CO3-AT3-Debugging and Optimization

View Only

← Back

QUESTIONS

Q1
ATM Withdrawal – Exception Handling Debugging
10 marks

Q2
Hospital Registration – NullPointerException Debugging
10 marks

Q3
E-Commerce Product Search – Array Exception Debugging
10 marks

Q4
Bank Account – Race Condition Debugging
10 marks

Q5
Railway Reservation – Synchronization Debugging
10 marks

5/5 answered

Q1 ATM Withdrawal – Exception Handling Debugging

10 marks

An ATM application is used to process cash withdrawals. The following Java program contains programming errors.

Task:

1. Run the given program and identify the errors.
2. Debug and modify the program.
3. Submit the COMPLETE corrected Java program.
4. The corrected program must handle insufficient balance, zero withdrawal, invalid input, and unexpected errors.
5. Use appropriate try-catch-finally exception handling.
6. The corrected program must execute successfully for all valid test cases.

Faulty Program:

```
import java.util.Scanner;

class ATM {
public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter account balance: ");
int balance = sc.nextInt();

System.out.print("Enter withdrawal amount: ");
int amount = sc.nextInt();

try {
int remaining = balance / amount;

if (amount > balance) {
```

```
if (amount > balance) {
System.out.println("Insufficient balance");
} else {
System.out.println("Withdrawal successful");
System.out.println("Remaining balance = "
+ (balance - amount));
}
}
catch (ArithmeticException e) {
System.out.println("Transaction failed");
}

System.out.println("Transaction completed");
}
}
```

Expected correction:
The program must correctly validate the withdrawal amount before performing the transaction, handle division-by-zero and invalid input, and always display "Transaction completed" after exception handling.

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

Test Input

Your Fixed Code

```java
import java.util.Scanner;

class ATM {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.println("Enter account balance: ");
            int balance = sc.nextInt();

            System.out.println("Enter withdrawal amount: ");
            int amount = sc.nextInt();

            if (amount <= 0) {
                throw new IllegalArgumentException("Withdrawal amount n
            }

            if (amount > balance) {
                throw new IllegalArgumentException("Insufficient balanc
            }

            int remaining = balance - amount;

            System.out.println("Withdrawal Successful");
            System.out.println("Remaining balance = " + remaining);

        } catch (java.util.InputMismatchException e) {
            System.out.println("Invalid input");

        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
```

Test Input

```
5000
2000
```

Output

```
Enter account balance:
Enter withdrawal amount:
Withdrawal Successful
Remaining balance = 3000
Transaction completed
```

Pending manual review

Pending manual review

```
            throw new IllegalArgumentException("Withdrawal amount must
        }

        if (amount > balance) {
            throw new IllegalArgumentException("Insufficient balance");
        }

        int remaining = balance - amount;

        System.out.println("Withdrawal Successful");
        System.out.println("Remaining balance = " + remaining);

    } catch (java.util.InputMismatchException e) {
        System.out.println("Invalid input");

    } catch (IllegalArgumentException e) {
        System.out.println(e.getMessage());

    } catch (Exception e) {
        System.out.println("Transaction failed");

    } finally {
        System.out.println("Transaction completed");
        sc.close();
    }
}
```

Submitted

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

Rayavaram Indhu
192524332RA

CO3-AT3-Debugging and Optimization

View Only

← Back

QUESTIONS

Q1 ✓
ATM Withdrawal – Exception Handling Debugging
10 marks

Q2 ✓
Hospital Registration – NullPointerException Debugging
10 marks

Q3 ✓
E-Commerce Product Search – Array Exception Debugging
10 marks

Q4 ✓
Bank Account – Race Condition Debugging
10 marks

Q5 ✓
Railway Reservation – Synchronization Debugging
10 marks

5/5 answered

Q2 Hospital Registration – NullPointerException Debugging

10 marks

A hospital patient registration application crashes when patient information is unavailable.

Debug the following complete Java program.

Task:

1. Identify the runtime error.
2. Modify the program to handle a missing patient name.
3. Handle invalid age input.
4. Use try-catch-finally appropriately.
5. The program must always display "Registration completed".
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
import java.util.Scanner;

class PatientRegistration {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter patient name: ");
String inputName = sc.nextLine();

String name;

if (inputName.equals("NULL")) {
name = null;
```

```
name = null;
} else {
name = inputName;
}

System.out.print("Enter age: ");
int age = sc.nextInt();

System.out.println("Patient Name: "
+ name.toUpperCase());

if (age >= 18) {
System.out.println("Adult patient");
} else {
System.out.println("Minor patient");
}

System.out.println("Registration completed");
}
}
```

The corrected program must prevent NullPointerException and handle invalid age input without terminating unexpectedly.

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2 import java.util.InputMismatchException;
3
4 class PatientRegistration {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
```

Test Input

```
Indhu
age
```

Output

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

Rayavaram Indhu
192524332RA

```
        try {
            System.out.println("Enter patient name:");
            String inputName = sc.nextLine();

            String name;

            if (inputName.equalsIgnoreCase("NULL") || inputName.trim()
                name = "Unknown";
            } else {
                name = inputName;
            }

            System.out.println("Enter age:");
            int age = sc.nextInt();

            if (age < 0) {
                System.out.println("Invalid age");
            } else {
                System.out.println("patient Name: " + name.toUpperCase

                if (age >= 18) {
                    System.out.println("Adult patient");
                } else {
                    System.out.println("Minor patient");
                }
            }

        } catch (InputMismatchException e) {
            System.out.println("Invalid age input");
```

```
Enter patient name:
Enter age:
Invalid age input
Registration completed
```

Pending manual review

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

Rayavaram Indhu
192524332RA

CO3-AT3-Debugging and Optimization

View Only

← Back

QUESTIONS

Q1
ATM Withdrawal – Exception Handling Debugging
10 marks

Q2
Hospital Registration – NullPointerException Debugging
10 marks

Q3
E-Commerce Product Search – Array Exception Debugging
10 marks

Q4
Bank Account – Race Condition Debugging
10 marks

Q5
Railway Reservation – Synchronization Debugging
10 marks

5/5 answered

Q3 E-Commerce Product Search – Array Exception Debugging

10 marks

An online shopping application allows customers to select a product using a product number from 1 to 5.

The following program contains an array indexing error.

Task:

1. Identify the error.
2. Correct the product-number and array-index logic.
3. Handle invalid product numbers.
4. Handle non-numeric input.
5. Submit the COMPLETE corrected Java program.
6. The program must display "Search completed" after processing.

Faulty Program:

```
import java.util.Scanner;

class ProductSearch {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

String[] products = {
"Laptop",
"Mobile",
"Tablet",
"Headphone",
```

```
};

System.out.print("Enter product number (1-5): ");
int productNumber = sc.nextInt();

System.out.println("Selected product: "
+ products[productNumber]);

System.out.println("Search completed");
}
}
```

Correct the program so that product numbers 1, 2, 3, 4 and 5 correctly access the corresponding products.

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1  import java.util.Scanner;
2
3  class ProductSearch {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          String[] products = {
8              "Laptop",
9              "Mobile",
10             "Tablet",
11             "Headphone",
12             "Keyboard"
13         };
14
15         try {
16             System.out.println("Enter product number (1-5):");
```

Test Input

```
1
```

Output

```
Enter product number (1-5):
Selected product: Laptop
Search completed
```

Pending manual review

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

Rayavaram Indhu
192524332RA

```
        "Keyboard"
    };

    try {
        System.out.println("Enter product number (1-5):");
        int productNumber = sc.nextInt();

        if (productNumber < 1 || productNumber > 5) {
            throw new IllegalArgumentException("Invalid product nu
        }

        System.out.println("Selected product: " + products[product
    } catch (java.util.InputMismatchException e) {
        System.out.println("Invalid input");

    } catch (IllegalArgumentException e) {
        System.out.println(e.getMessage());

    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Invalid product number");

    } catch (Exception e) {
        System.out.println("Search failed");

    } finally {
        System.out.println("Search completed");
        sc.close();
    }
  }
}
```

Pending manual review

CO3-AT3-Debugging and Optimization

View Only

← Back

QUESTIONS

5/5 answered

Q4 Bank Account – Race Condition Debugging

10 marks

A bank account contains ₹10,000. Two ATM threads simultaneously attempt to withdraw ₹7,000 and ₹6,000.

The following program contains a race condition.

Task:

1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class BankAccount {

int balance = 10000;

void withdraw(int amount) {

if (balance >= amount) {

System.out.println(
Thread.currentThread().getName()
+ " withdrawing " + amount);

try {
Thread.sleep(100);
}
```

```
}
catch (InterruptedException e) {
System.out.println("Interrupted");
}

balance = balance - amount;

System.out.println(
Thread.currentThread().getName()
+ " withdrawal successful");
}
else {
System.out.println(
Thread.currentThread().getName()
+ " insufficient balance");
}
}
}

class ATM extends Thread {

BankAccount account;
int amount;

ATM(BankAccount account, int amount) {
this.account = account;
this.amount = amount;
}

public void run() {
account.withdraw(amount);
}
```

```
class BankApplication {

public static void main(String[] args)
throws InterruptedException {

BankAccount account = new BankAccount();

ATM atm1 = new ATM(account, 7000);
ATM atm2 = new ATM(account, 6000);

atm1.setName("ATM-1");
atm2.setName("ATM-2");

atm1.start();
atm2.start();

atm1.join();
atm2.join();

System.out.println(
"Final Balance = " + account.balance);
}
}
```

Correct the program so that only one withdrawal succeeds.

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 class BankAccount {
2     int balance = 10000;
3
4     synchronized void withdraw(int amount, String name) {
5         if (balance >= amount) {
```

Test Input

stdin input (optional)

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

Rayavaram Indhu
192524332RA

```
            System.out.println(name + " withdrawing " + amount);

            balance = balance - amount;

            System.out.println(name + " withdrawal successful");
        } else {
            System.out.println(name + " insufficient balance");
        }
    }

    public static void main (String[] args){
        BankAccount account = new BankAccount();

        Thread t1 = new Thread(new Runnable(){
            public void run(){
                account.withdraw(7000, "ATM-1");
            }
        });

        Thread t2 = new Thread(new Runnable(){
            public void run() {
                account.withdraw(6000, "ATM-2");
            }
        });

        t1.start();
        t2.start();
    }
}
```

Output

```
ATM-1 withdrawing 7000
ATM-1 withdrawal successful
ATM-2 insufficient balance
```

Pending manual review

Submitted

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

Rayavaram Indhu
192524332RA

CO3-AT3-Debugging and Optimization

View Only

← Back

QUESTIONS

Q1
ATM Withdrawal – Exception Handling Debugging
10 marks

Q2
Hospital Registration – NullPointerException Debugging
10 marks

Q3
E-Commerce Product Search – Array Exception Debugging
10 marks

Q4
Bank Account – Race Condition Debugging
10 marks

Q5
Railway Reservation – Synchronization Debugging
10 marks

5/5 answered

Q5 Railway Reservation – Synchronization Debugging

10 marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class Reservation {

int seats = 1;

void bookSeat(String customer) {

if (seats > 0) {

System.out.println(customer
+ " is booking a seat");

try {
Thread.sleep(100);
}
catch (InterruptedException e) {
```

```
catch (InterruptedException e) {
System.out.println("Thread interrupted");
}
seats--;
System.out.println(customer
+ " booking successful");
} else {
System.out.println(customer
+ " - No seat available");
}
}
}
class Customer extends Thread {
Reservation reservation;
String customerName;
Customer(Reservation reservation, String customerName) {
this.reservation = reservation;
this.customerName = customerName;
}
public void run() {
reservation.bookSeat(customerName);
}
}
class RailwayReservation {
public static void main(String[] args) {
Reservation reservation = new Reservation();
```

```
Reservation reservation = new Reservation();

Customer c1 =
new Customer(reservation, "Customer 1");

Customer c2 =
new Customer(reservation, "Customer 2");

c1.start();
c2.start();
}
}
```

The corrected program must protect the shared seat resource using synchronization.

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1  class Reservation {
2      int seats = 1;
3
4      synchronized void bookSeat(String customer) {
5          if (seats > 0) {
6              System.out.println(customer + " is booking a seat");
7
8              seats = seats - 1;
9
10             System.out.println(customer + " booking successful");
11         } else {
12             System.out.println(customer + " - No seat available");
13         }
14     }
15 public static void main(String[] args) {
```

Test Input

```
stdin input (optional)
```

Output

```
Customer 1 is booking a seat
Customer 1 booking successful
Customer 2 - No seat available
```

Pending manual review

SIMATS ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

Rayavaram Indhu
192524332RA

```
    synchronized void bookSeat(String customer) {
        if (seats > 0) {
            System.out.println(customer + " is booking a seat");

            seats = seats - 1;

            System.out.println(customer + " booking successful");
        } else {
            System.out.println(customer + " - No seat available");
        }
    }
public static void main(String[] args) {
    Reservation reservation = new Reservation();

    Thread t1 = new Thread(new Runnable(){
        public void run() {
            reservation.bookSeat("Customer 1");
        }
    });

    Thread t2 = new Thread (new Runnable(){
        public void run() {
            reservation.bookSeat("Customer 2");
        }
    });
    t1.start();
    t2.start();
}
}
```

Output

```
Customer 1 is booking a seat
Customer 1 booking successful
Customer 2 - No seat available
```

Pending manual review

Submitted